

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4382

Калинина А.В
Смирнов Н.А
Бурая В.С.
Писаренко А.А.

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: А.А. Писаренко

Группа: 4382

Тема практики: UI-тестирование

Задание на практику:

Задание заключается в реализации полного цикла автоматизированного тестирования функциональности приложения с имитацией действий пользователя. Работа включает в себя проектирование сценариев для пяти различных функций приложения, их согласование с заказчиком (преподавателем), написание автотестов и подготовку сопутствующей документации. Реализация проекта осуществляется на языке Java с использованием инструментов Maven, JUnit и Selenium/Selenide.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты	_____	Калинина А.В. Смирнов Н.А. Бурая В.С. Писаренко А.А.
Руководитель	_____	А.М. Шевелёва

АННОТАЦИЯ

Работа посвящена созданию гибкой фреймворк-архитектуры для автоматизации UI-тестирования онлайн-платформы Stepik. В процессе выполнения проекта исследованы ключевые возможности системы, на основе чего сформирован набор из 10 функциональных тест-кейсов. Программная база реализована на Java с применением паттерна Page Object Model, что позволило чётко разделить логику проверок от элементов интерфейса. В стек технологий вошли Selenide, JUnit 5 и средство регистрации событий Log4j2. Проект снабжён технической документацией, содержащей UML-диаграммы классов, и опубликован в репозитории на GitHub.

SUMMARY

This work is dedicated to creating a flexible framework architecture for automated UI testing of the Stepik online platform. During the project, the key capabilities of the system were analyzed to build a set of 10 functional test cases. The codebase was implemented in Java using the Page Object Model pattern, enabling a clear separation of test logic from user interface elements. The technology stack includes Selenide, JUnit 5, and the Log4j2 logging tool. The project is supported by technical documentation, including UML class diagrams, and is published in a GitHub repository.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
2 РЕЗУЛЬТАТЫ РАБОТЫ	24
2.1. Описание программных классов и их методов.....	24
2.2. UML-диаграмма классов.....	26
2.3. Реализация взаимодействия компонентов	27
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	29
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	32
ЗАКЛЮЧЕНИЕ	33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35

ВВЕДЕНИЕ

Предметной областью данной практики является разработка и внедрение автоматизированных сценариев тестирования для веб-приложений, что позволяет ускорить процесс проверки функционала, минимизировать влияние человеческого фактора и обеспечить стабильность работы поисковых модулей современных информационных систем.

Целью практики является реализация полного цикла автоматизированного тестирования функциональности веб-приложения (на примере модуля поиска) с использованием стека технологий Java. В ходе работы необходимо было спроектировать архитектуру проекта, применить современные паттерны проектирования и обеспечить полное покрытие ключевых пользовательских сценариев автоматизированными тестами.

Задачи практики включают в себя: развертывание инфраструктуры проекта и настройку среды окружения; проектирование объектной модели страниц приложения для разделения логики тестов и интерфейса; разработку десяти функциональных автотестов, охватывающих позитивные и негативные сценарии взаимодействия с системой; а также подготовку отчетной технической документации, включая чек-листы и инструкции по запуску.

Вклад каждого участника:

Калинина А.В.: реализация Page Objects (HomePage, SearchResultsPage, CoursePage, ProfilePages) [1], создание чек-листа, настройка логирования, подготовка и финализация README, приведение классов к единому порядку, вынесение Selenide [2] элементов в переменные, удаление неиспользуемых методов и импортов, добавление документации к классам pages и BaseTest.

Смирнов Н.А.: Архитектура проекта и базовая настройка: создание Maven проекта, подключение зависимостей (Selenide, JUnit5 [3], Log4j2 [4]), реализация папки components (SearchComponent, FilterComponent, CourseCardItem и т.д.), настройка BaseTest, инициализация структуры проекта, работа над чек-листом, проверка архитектуры кода и проверка соблюдения принципов Page Object Model, приведение классов к единому порядку,

вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов.

Бурая В.С.: Тесты №1–5: навигация по страницам, поиск с фильтрами, фильтрация по ценовому диапазону, поиск для продвинутых с сертификатом, сброс текстового поиска "крестиком". Проверка стабильности выполнения тестов. Исправление и добавление методов. Удаление всех комментариев с действиями из тестов. Вынесение тестов с фильтрацией в отдельный класс.

Писаренко А.А.: Тесты №6–10: смена запроса с сохранением фильтра, несуществующий запрос с фильтрами, поиск по автору с переходом в профиль, фильтрация акционных курсов, добавление курса в избранное из поиска. Добавление assert-проверок и обработка исключений. Проверка работы всех тестов в связке. Исправление и добавление некоторых методов. Создание отдельного класса для работы с окнами. Вынесение всех магических чисел в константы.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 10 тестов:

Текстовый поиск и навигация по страницам

Поиск с фильтрами

Фильтрация по ценовому диапазону

Поиск для продвинутых с сертификатом

Сброс текстового поиска «крестиком»

Смена запроса с сохранением фильтра

Несуществующий запрос с фильтрами

Поиск по автору с переходом в профиль

Фильтрация акционных курсов

Добавление курса в избранное из поиска

Тест №1: Текстовый поиск и навигация по страницам

Проверка корректности поиска, по ключевым словам, работы переключения страниц результатов (пагинации) и перехода на страницу с детальной информацией о курсе.

Результаты действий представлены на рисунках (см. рисунки 1 – 4).

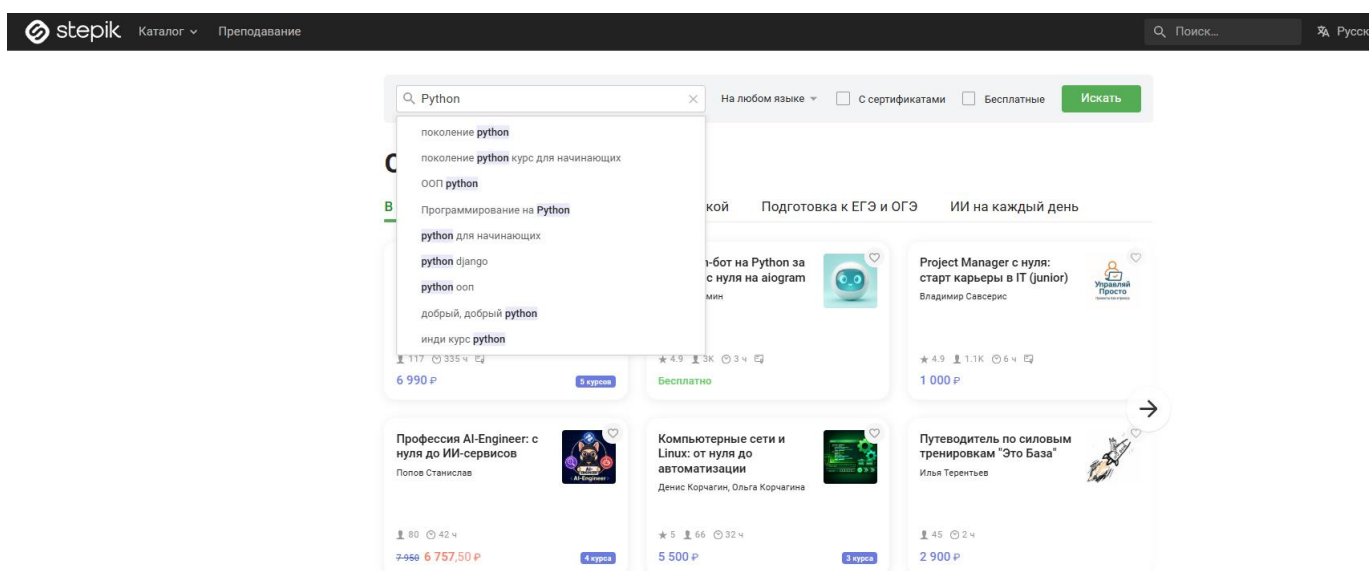


Рисунок 1 – Введение в поиск запрос «Python»

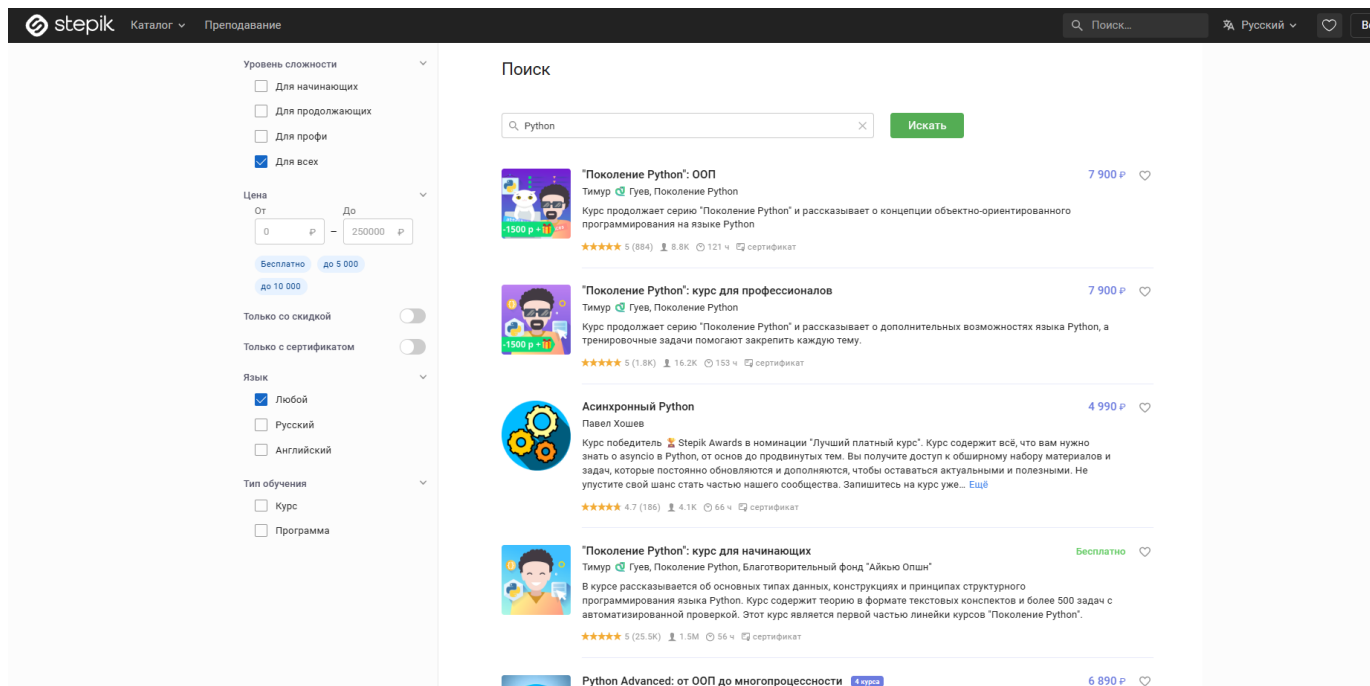


Рисунок 2 – Результаты поиска по запросу «Python»

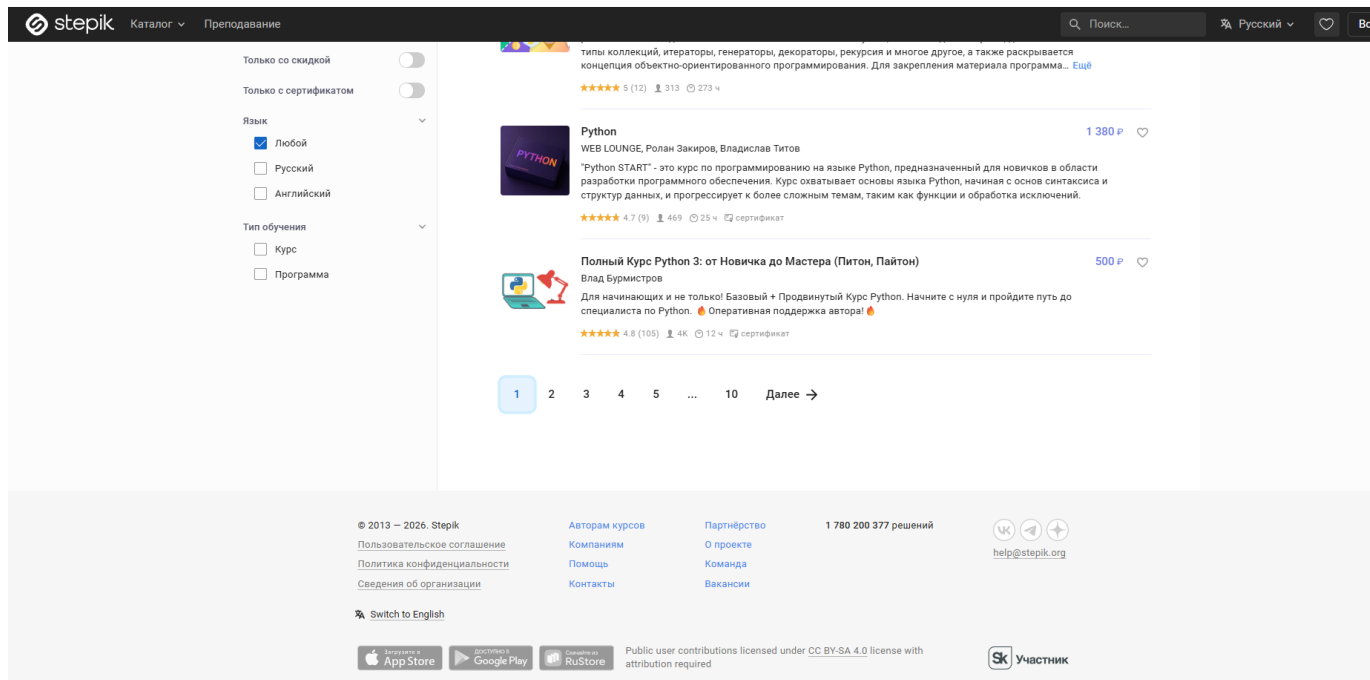


Рисунок 3 – Прокрутка вниз к пагинации

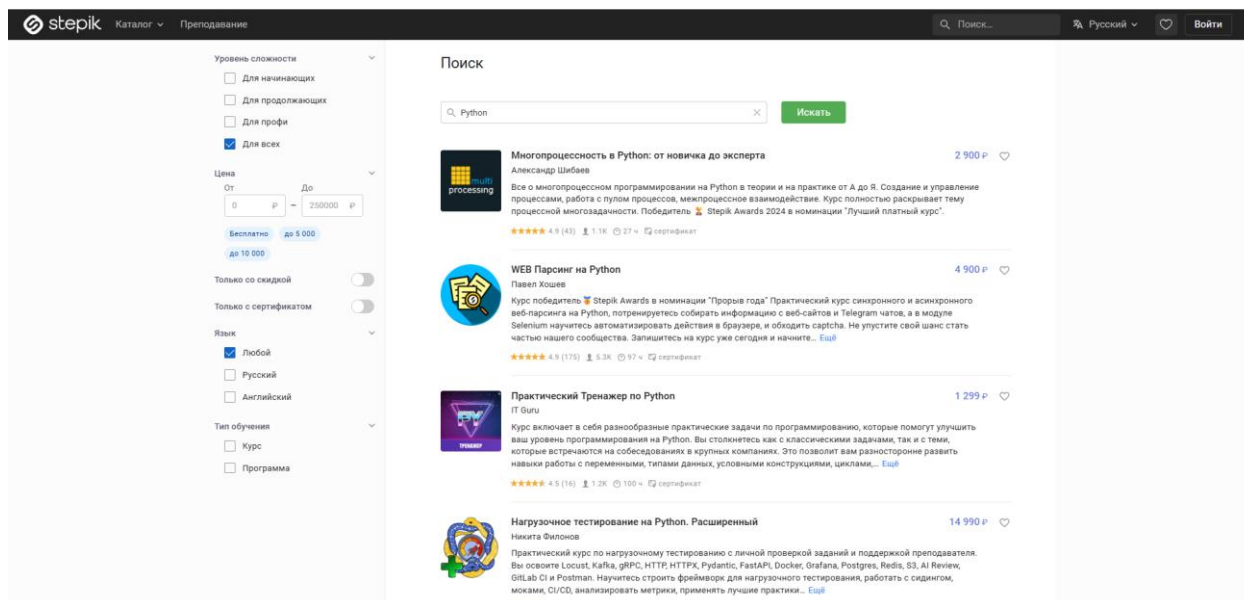


Рисунок 4 – Переход на вторую страницу запросов

Тест №2: Поиск с фильтрами

Валидация работы системы фильтрации: проверка точного отсеивания результатов по выбранному языку обучения и типу курса.

Результаты действий представлены на рисунках (см. рисунки 5, 6).

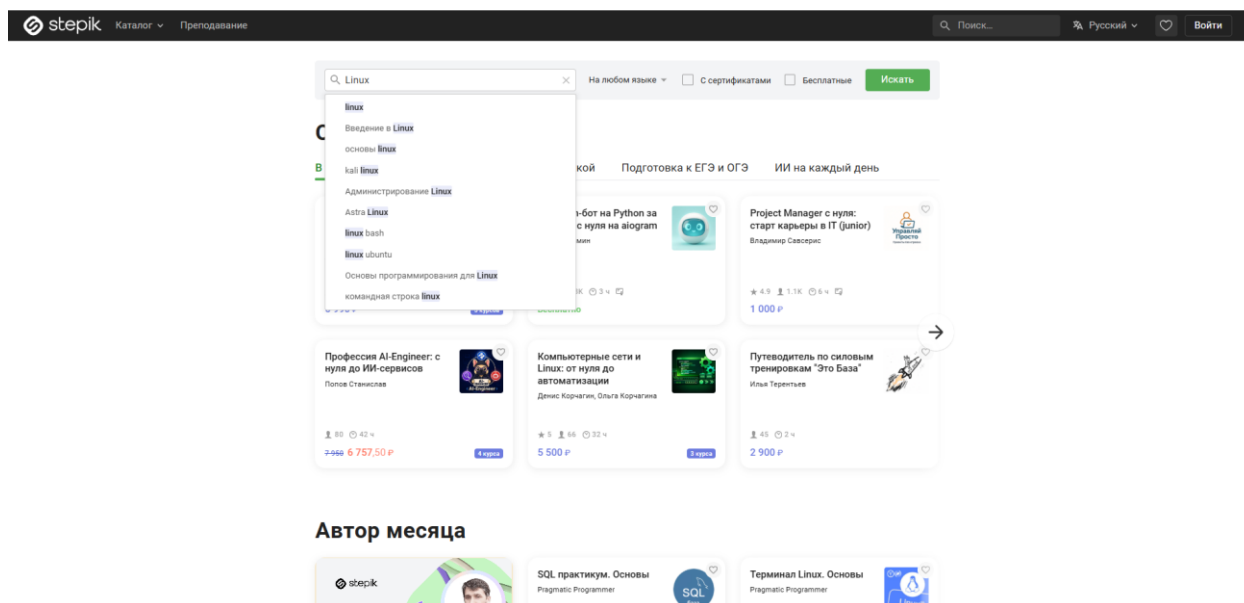


Рисунок 5 – Введение в поиск запрос «Linux»

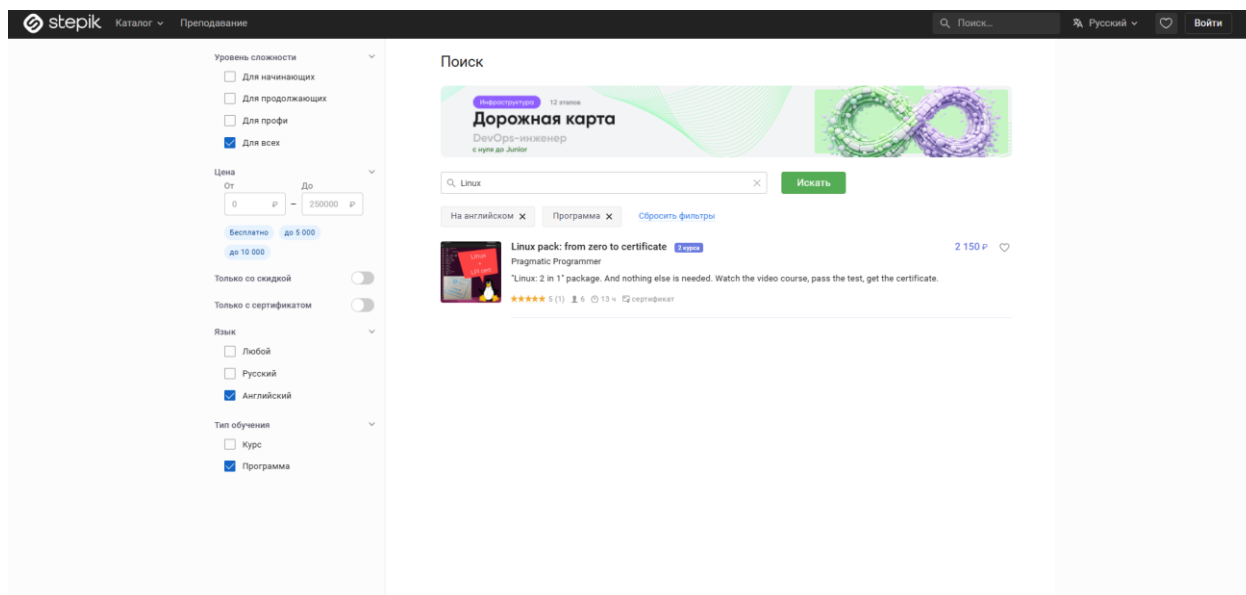


Рисунок 6 – Применение чекбоксов «Английский» и «Программа»

Тест №3: Фильтрация по ценовому диапазону

Проверка корректности отбора курсов, попадающих строго в заданные границы стоимости, через поля ввода минимальной и максимальной цены.

Результаты действий представлены на рисунках (см. рисунки 7, 8).

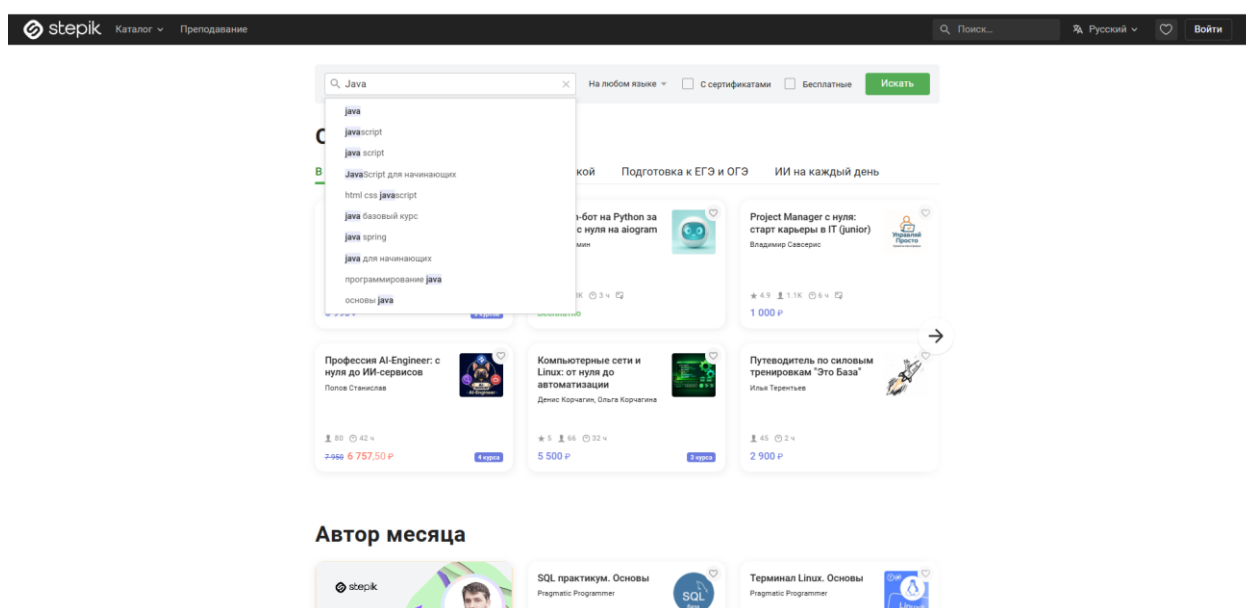


Рисунок 7 – Введение в поиск запрос «Java»

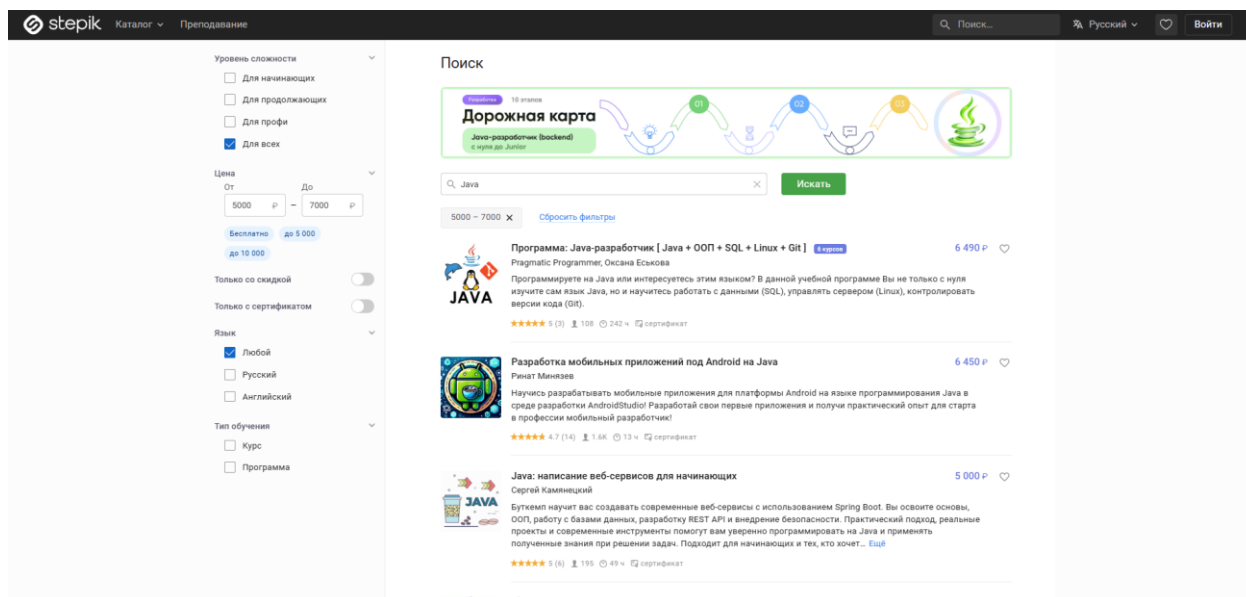


Рисунок 8 – Применение фильтра по цене

Тест №4: Поиск для продвинутых с сертификатом

Тестирование одновременного применения фильтра «продвинутый уровень сложности» и переключателя «только курсы с сертификатом».

Результаты действий представлены на рисунках (см. рисунки 9 – 10).

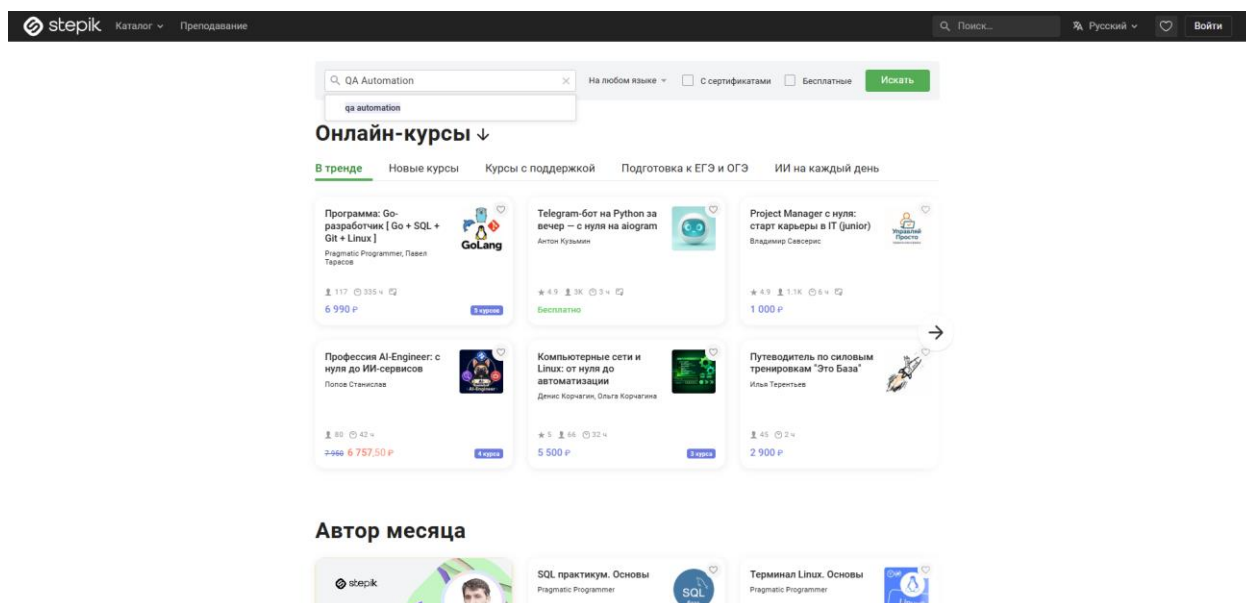


Рисунок 9 – Введение в поиск запрос «QA Automation»

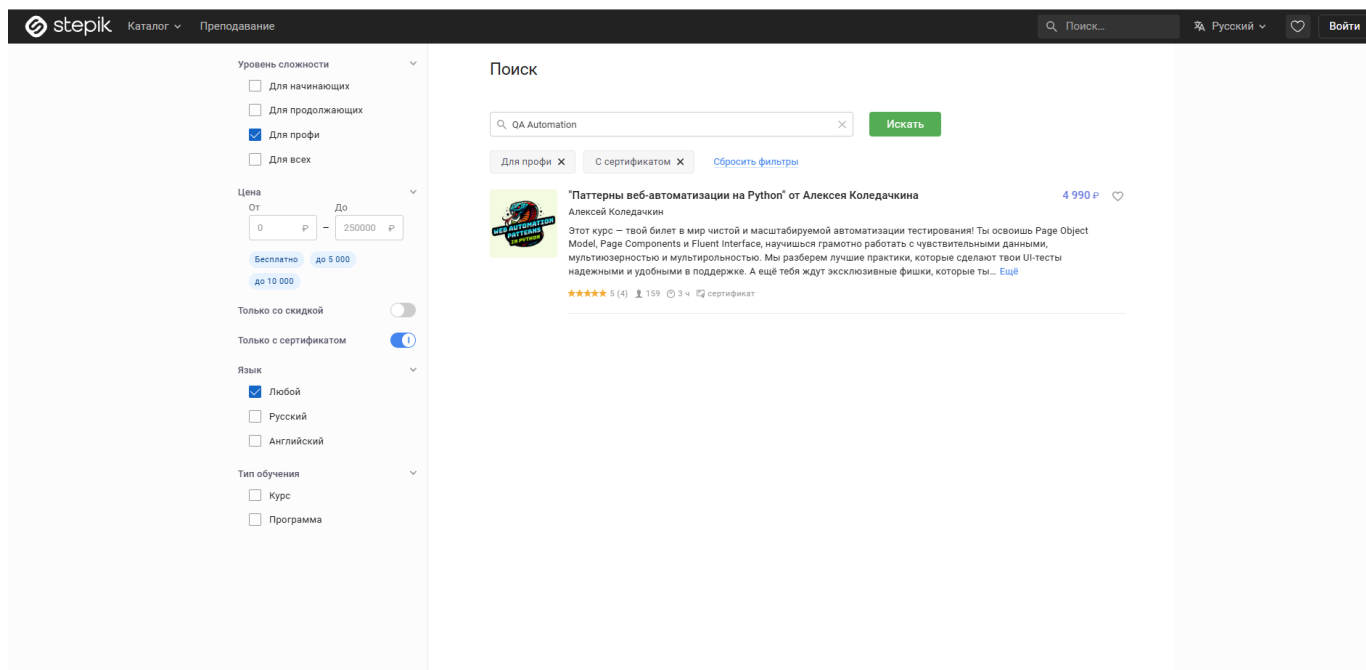


Рисунок 10 – Применение чекбоксов «Для профи» и «Только с сертификатом»

Тест №5: Сброс текстового поиска «крестиком»

Проверка работы кнопки очистки (крестик) в поисковой строке: полное удаление введенного текста и автоматический сброс выдачи к исходному состоянию каталога.

Результаты действий представлены на рисунках (см. рисунки 11 - 13).

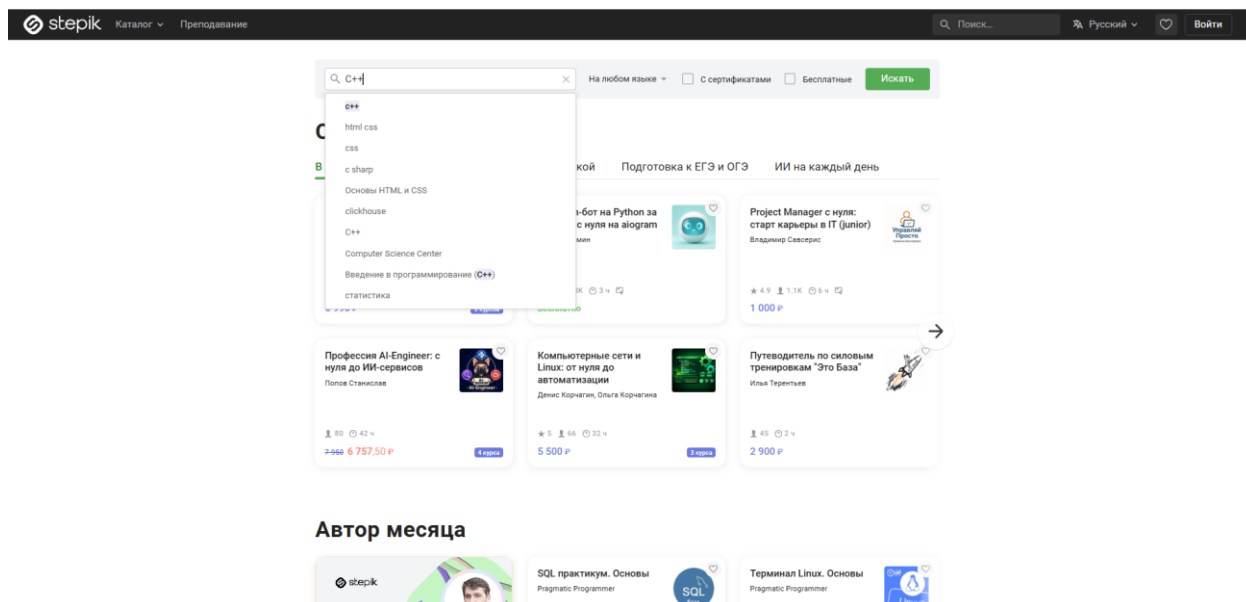


Рисунок 11 – Введение в поиск запрос «C++»

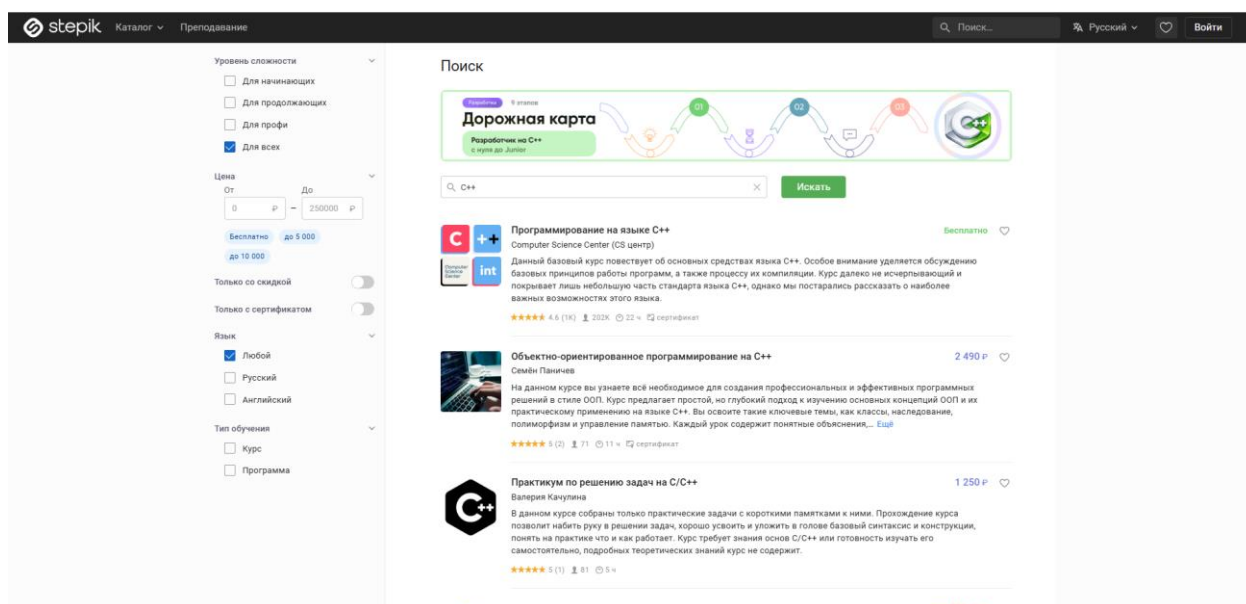


Рисунок 12 – Вывод всех доступных курсов

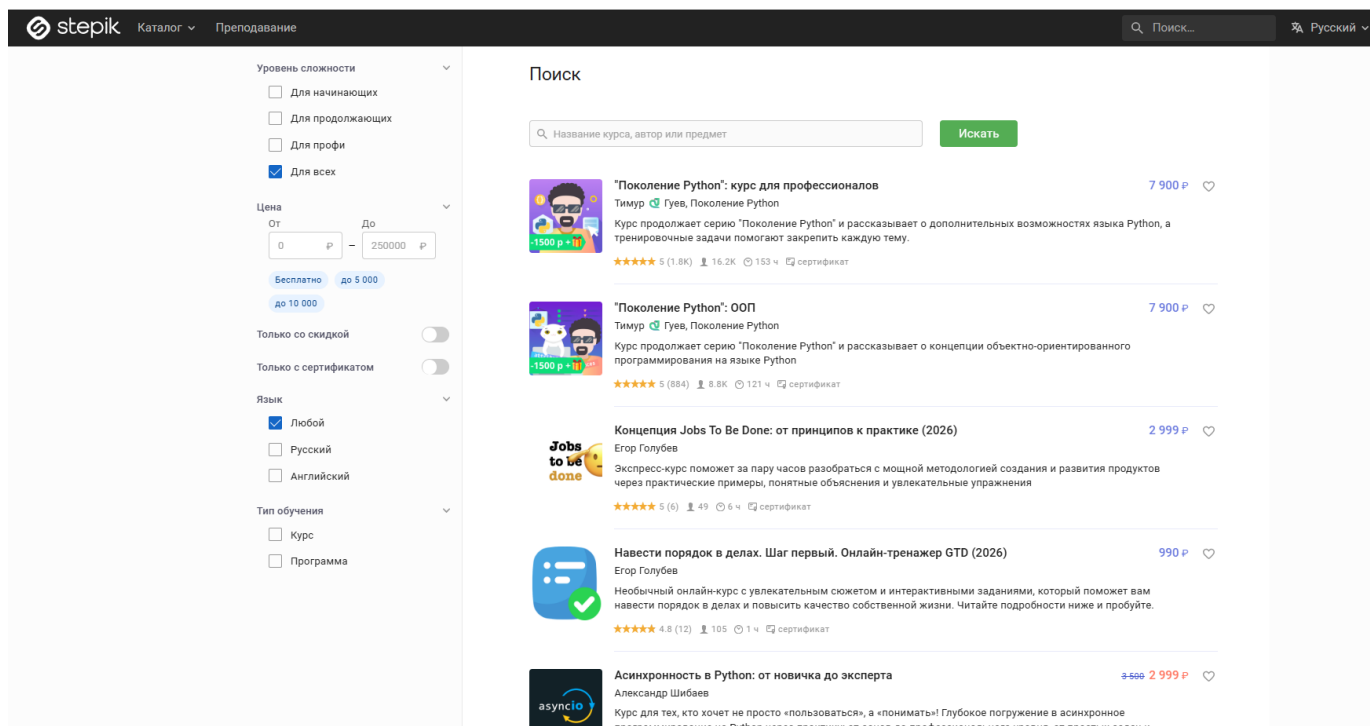


Рисунок 13 – Нажатие крестика

Тест №6: Смена запроса с сохранением фильтра

Проверка того, что ранее выбранные фильтры (например, язык обучения) остаются активными при вводе нового ключевого слова в поисковую строку.

Результаты действий представлены на рисунках (см. рисунки 14 – 16).

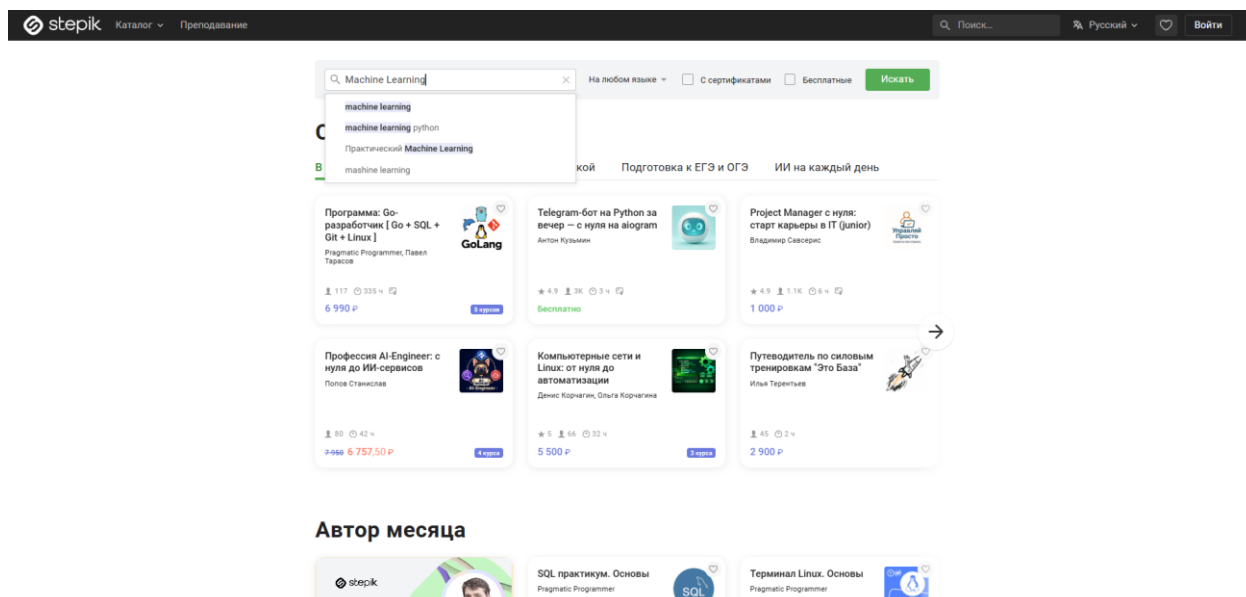


Рисунок 14 – Введение в поиск запрос «Machine Learning»

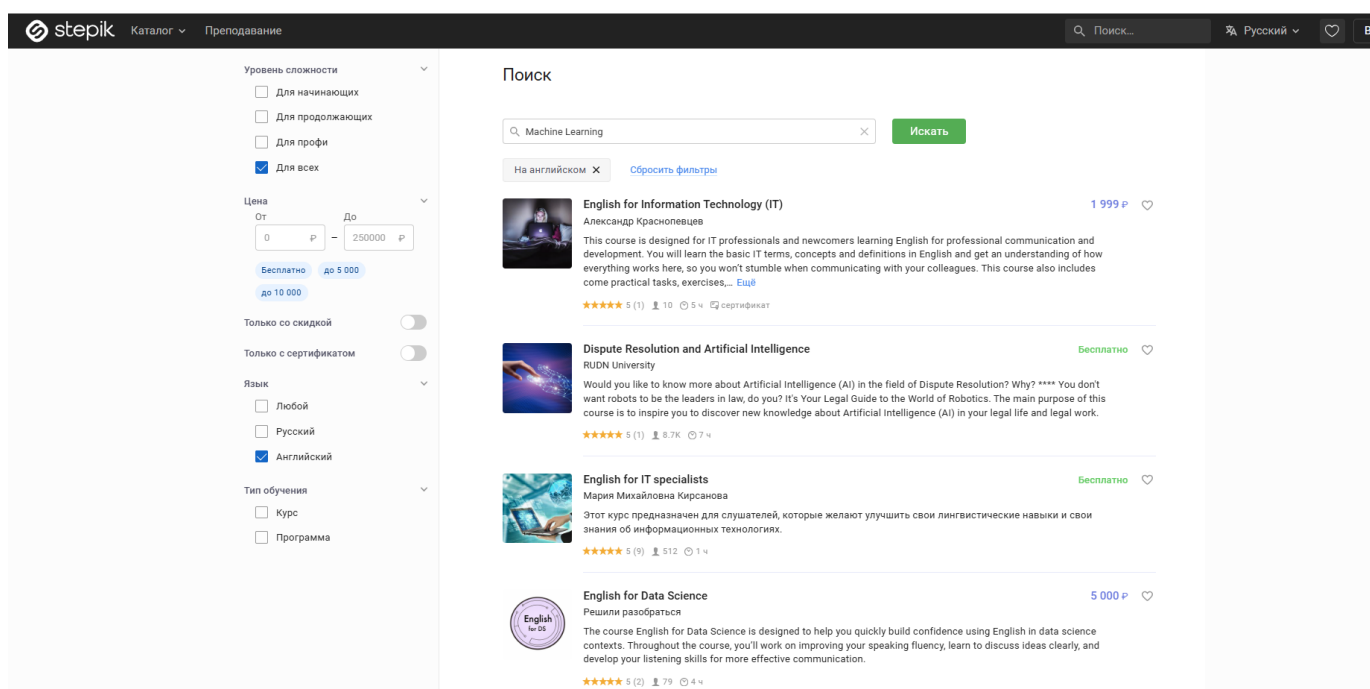


Рисунок 15 – Показ всех доступных курсов

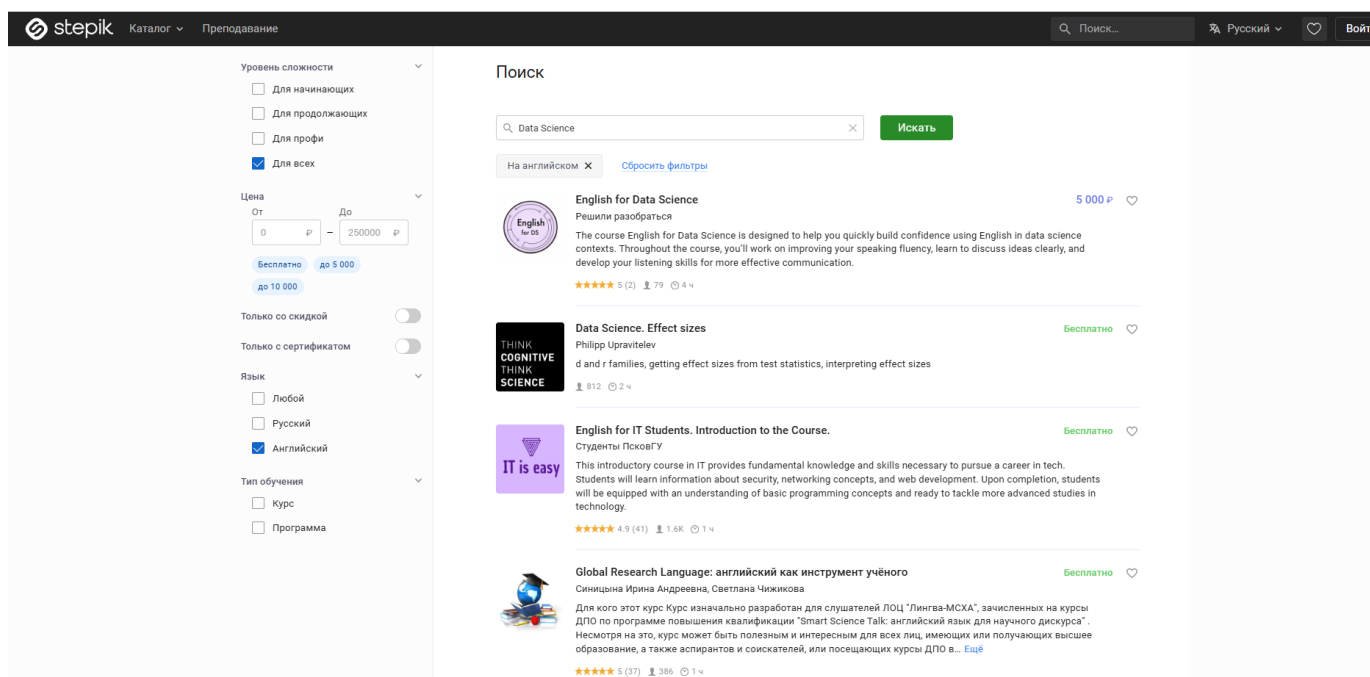


Рисунок 16 – Введение нового запроса «Data Science»

Тест №7: Несуществующий запрос с фильтрами

Проверка негативного сценария: корректное отображение сообщения «Ничего не найдено» при вводе некорректного или бессмысленного запроса с активными фильтрами.

Результаты действий представлены на рисунках (см. рисунки 17, 18).

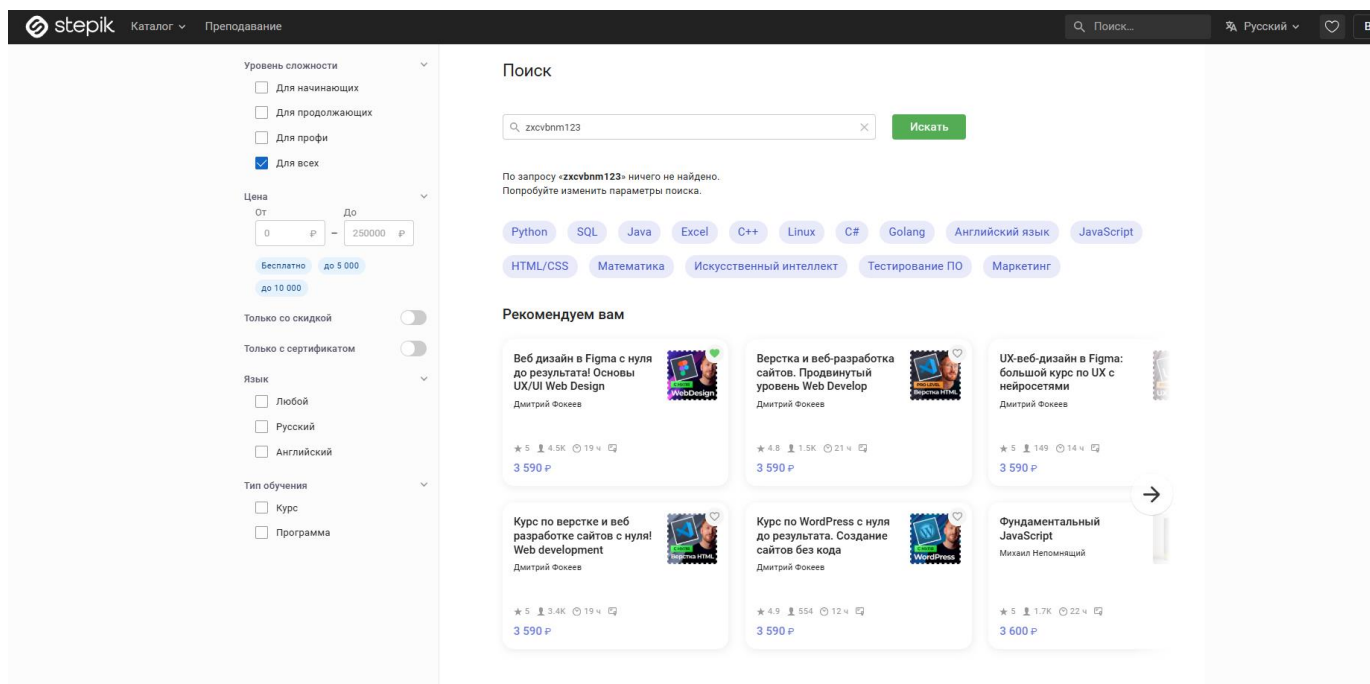


Рисунок 17 – Введение в поиск запрос «ячсми123»

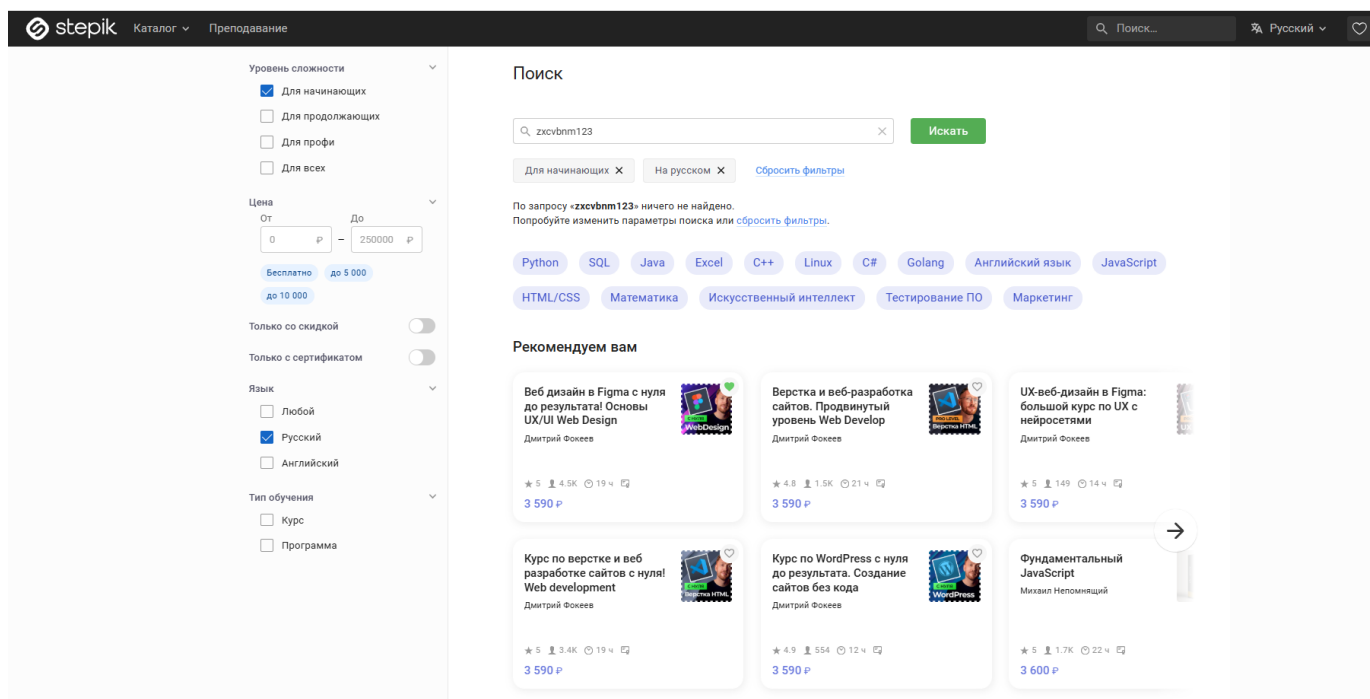


Рисунок 18 – Применение чекбоксов «Русский» и «Для начинающих»

Тест №8: Поиск по автору с переходом в профиль

Валидация точности поиска курсов по имени автора и работоспособности ссылки для перехода из карточки курса в личный профиль преподавателя.

Результаты действий представлены на рисунках (см. рисунки 19 – 22).

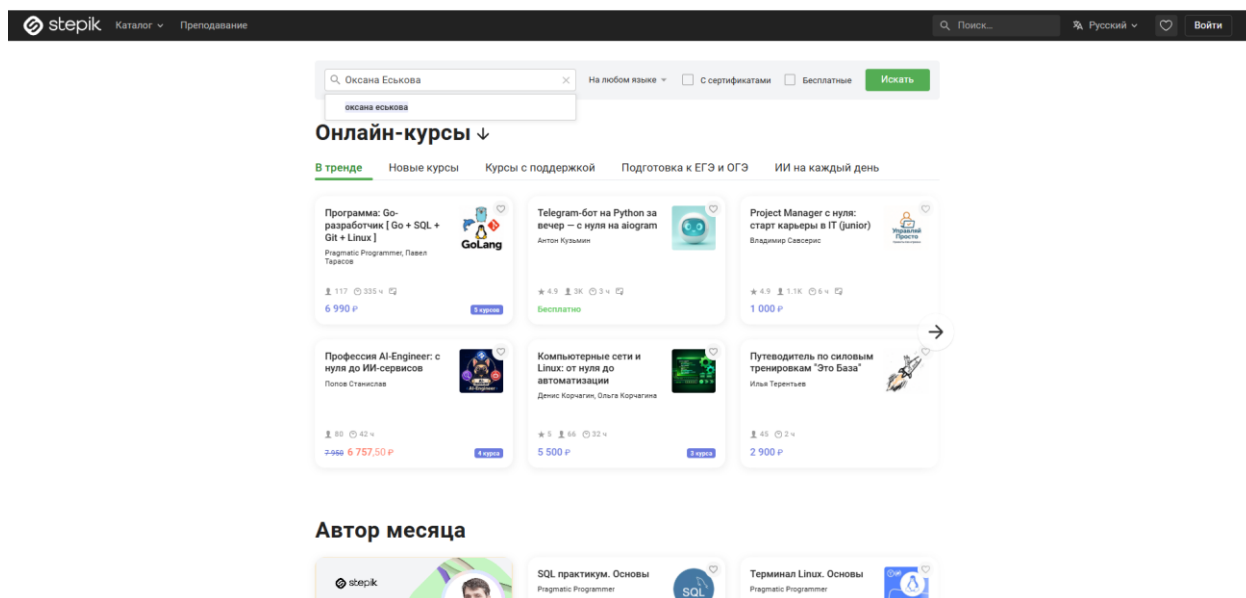


Рисунок 19 – Введение в поиск запрос «Оксана Еськова»

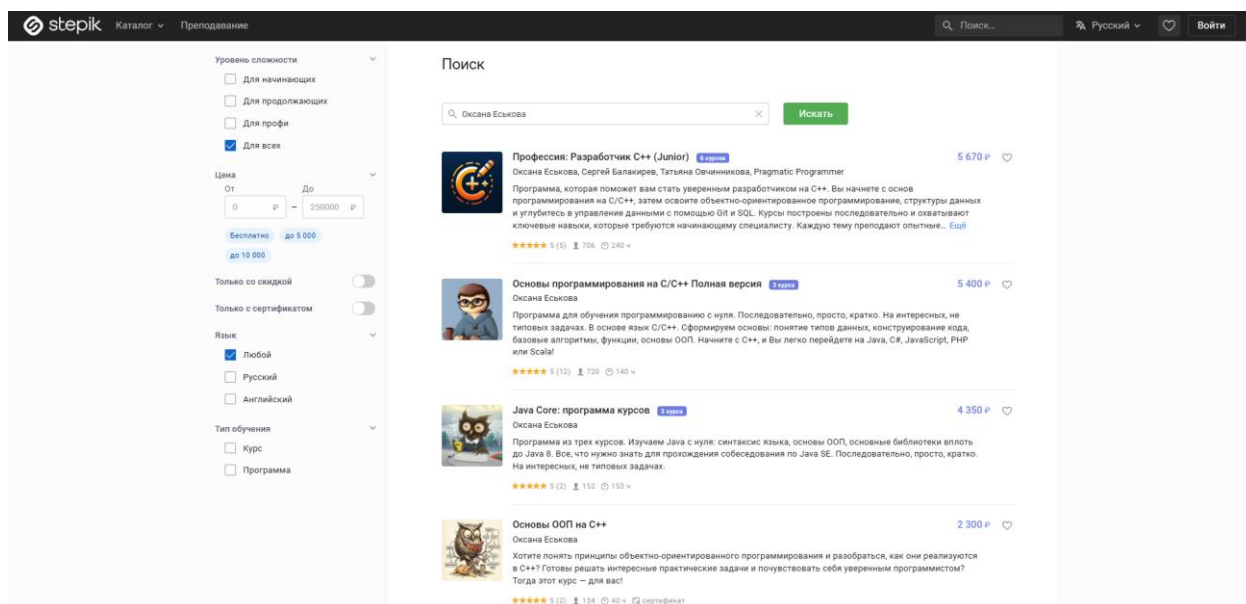


Рисунок 20 – Показ всех доступных курсов

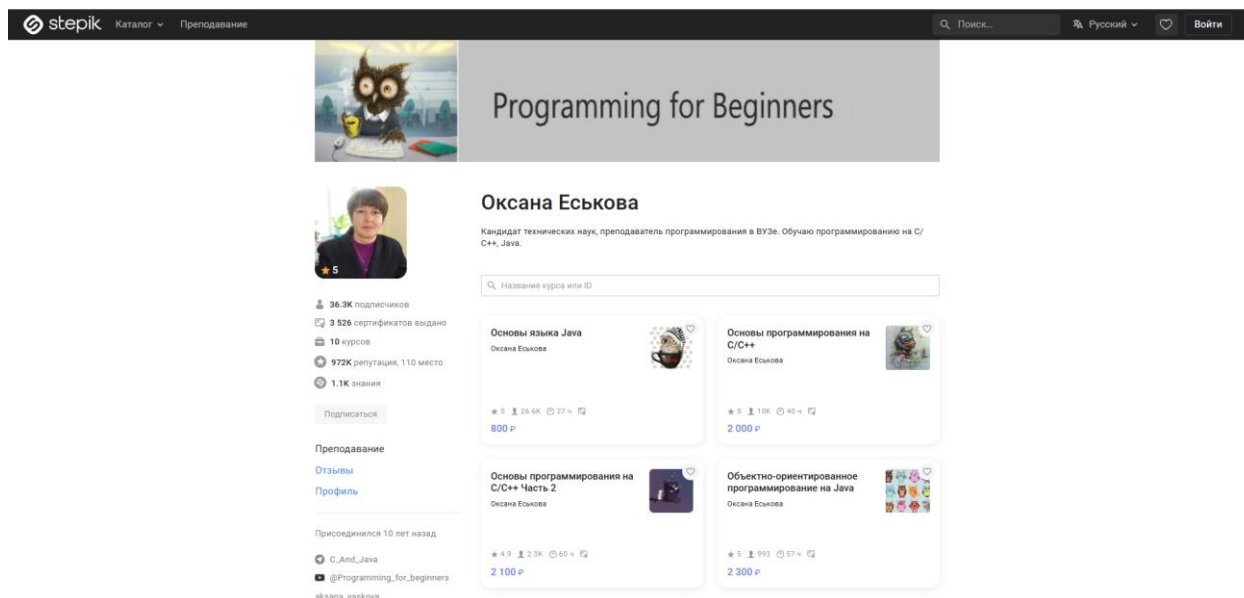


Рисунок 21 – Переход в профиль пользователя «Оксана Еськова»

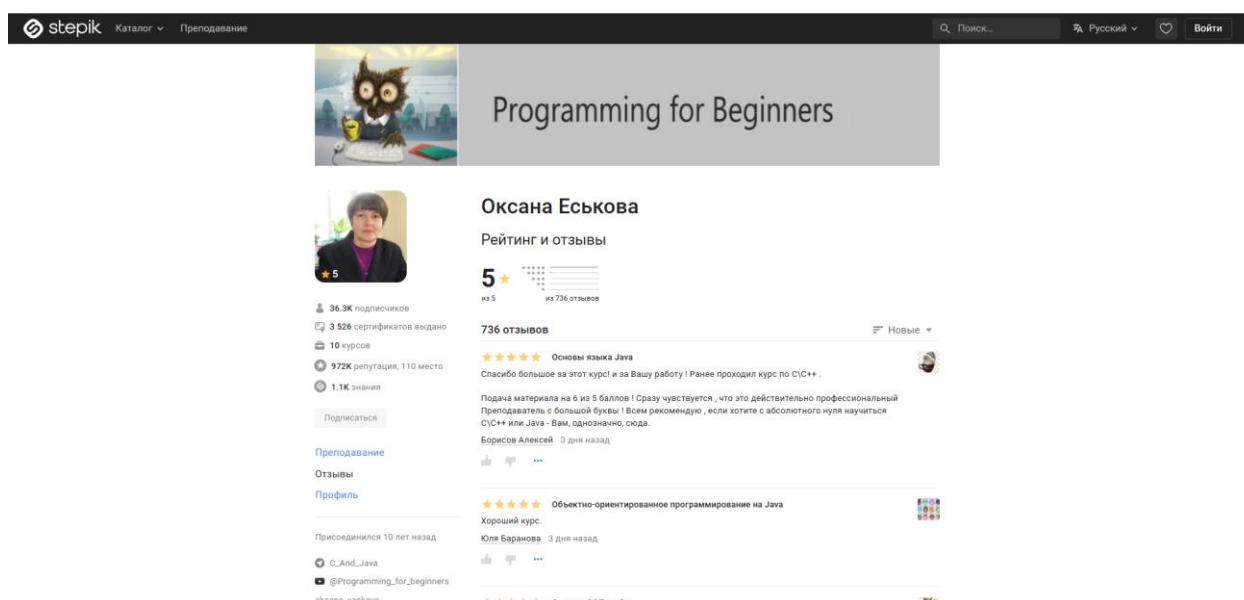


Рисунок 22 – Переход во вкладку «Отзывы»

Тест №9: Фильтрация акционных курсов

Проверка работы переключателя «со скидкой»: в результатах выдачи должны отображаться только те карточки, где визуально присутствует зачеркнутая старая цена.

Результаты действий представлены на рисунках (см. рисунки 23, 24).

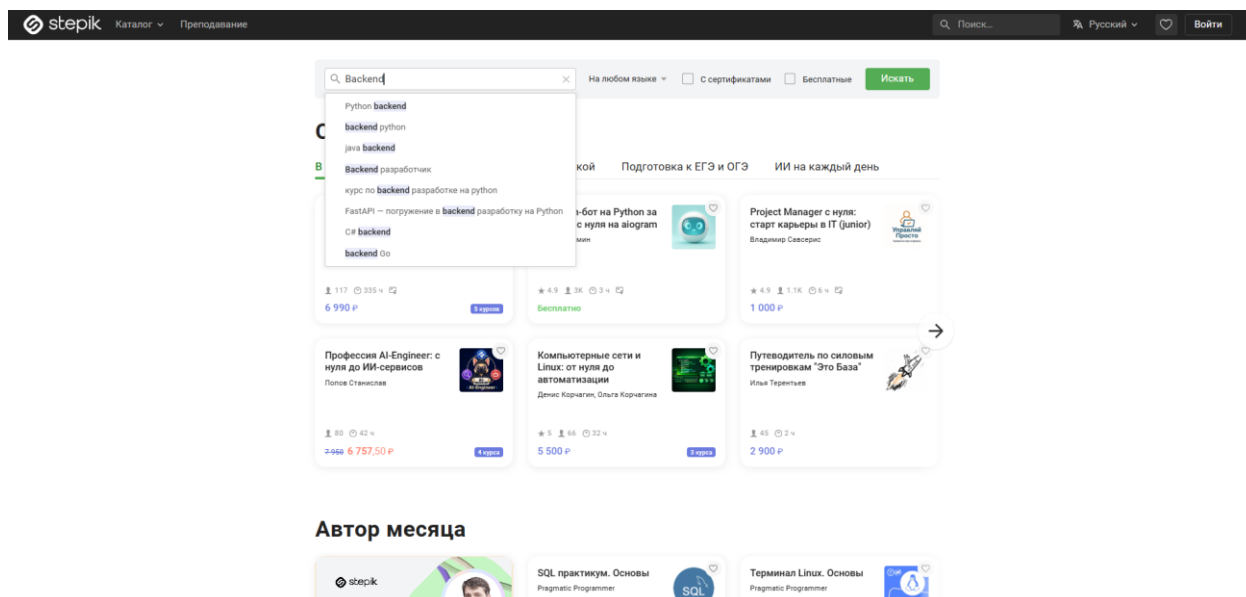


Рисунок 23 – Введение в поиск запрос «Backend»

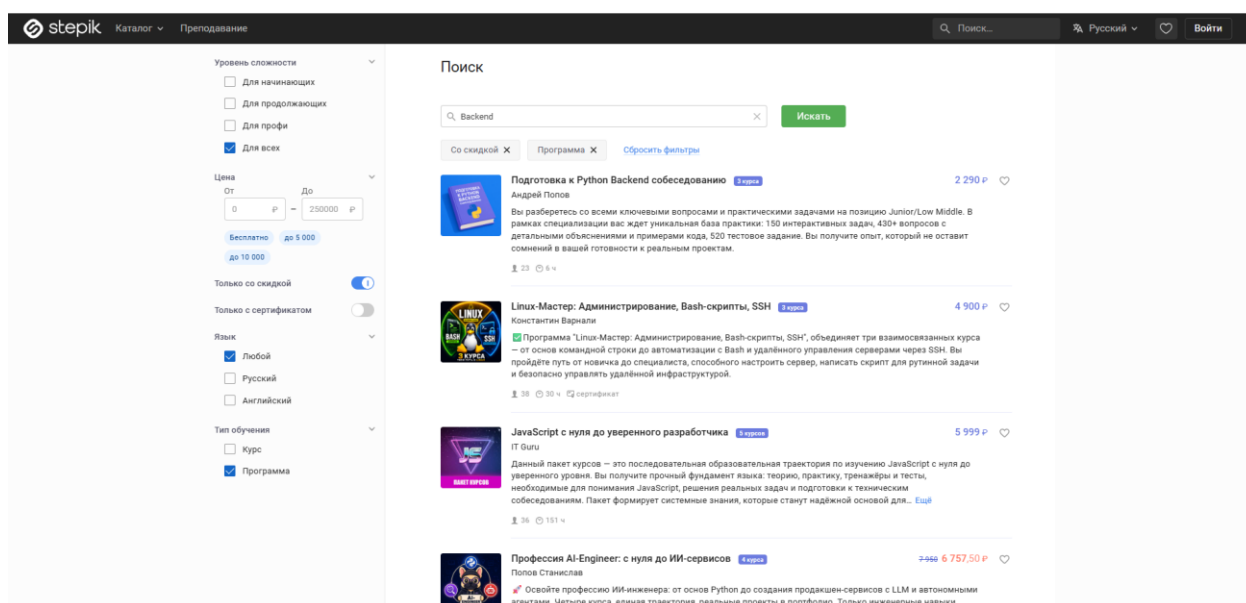


Рисунок 24 – Применение чекбоксов «Только со скидкой» и «Программа»

Тест №10: Добавление курса в избранное из поиска

Тестирование интерактивности карточки курса: возможность добавить курс в список желаемого прямо из выдачи и мгновенное визуальное изменение состояния иконки «сердечко».

Результаты действий представлены на рисунках (см. рисунки 25 – 27).

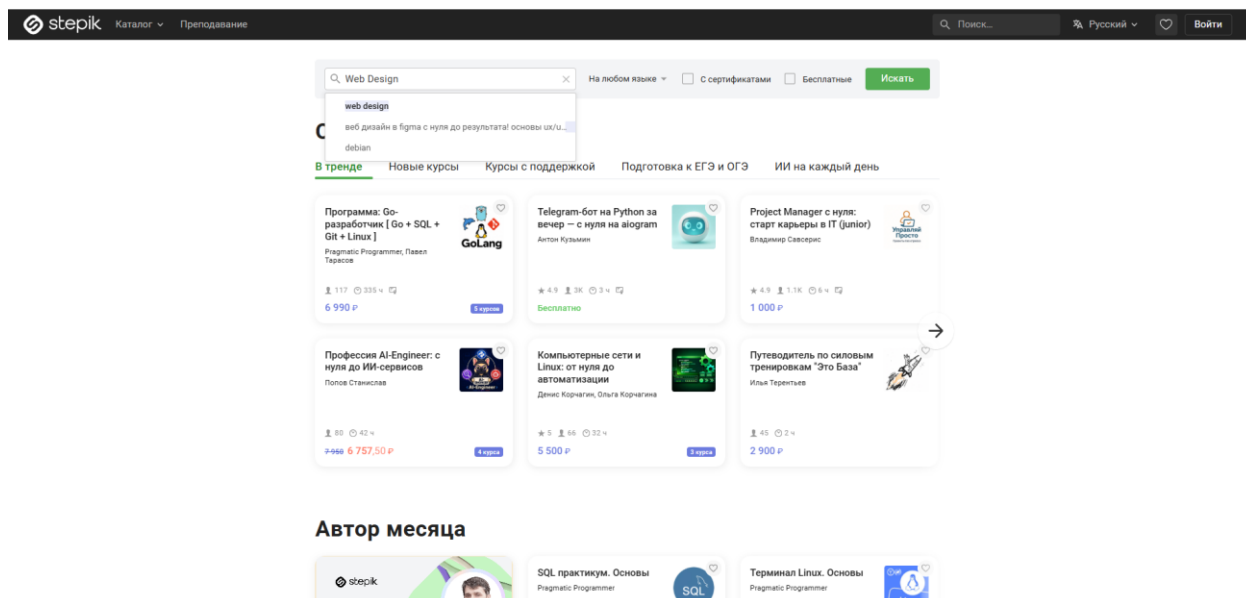


Рисунок 25 – Введение в поиск запрос «Web Design»

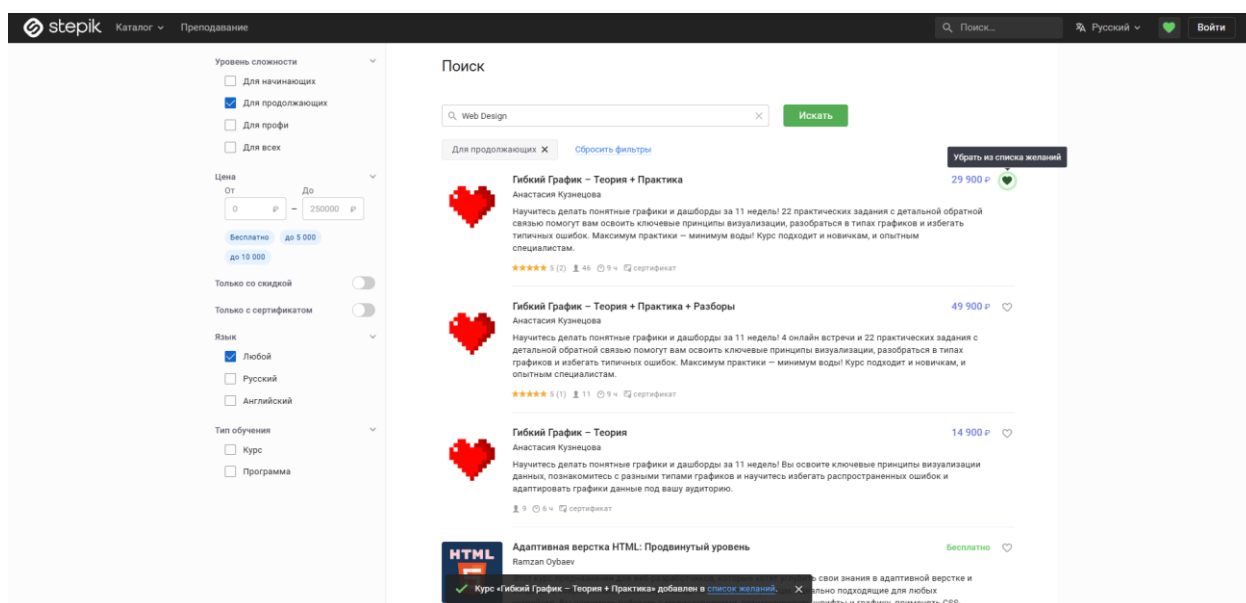


Рисунок 26 – Добавление первого курса в избранное

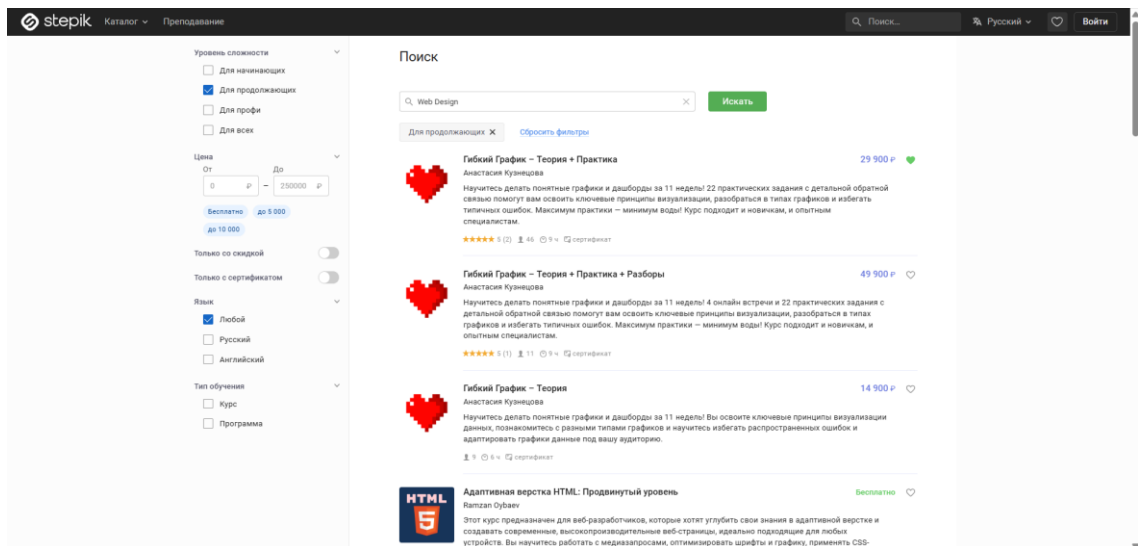


Рисунок 27 – Показ того, что первый курс в избранном (сердечко стало закрашенным)

Тесты поделены на два блока: поиск и фильтрация.

Результаты выполнения первой группы тестов представлены на рис. 28:

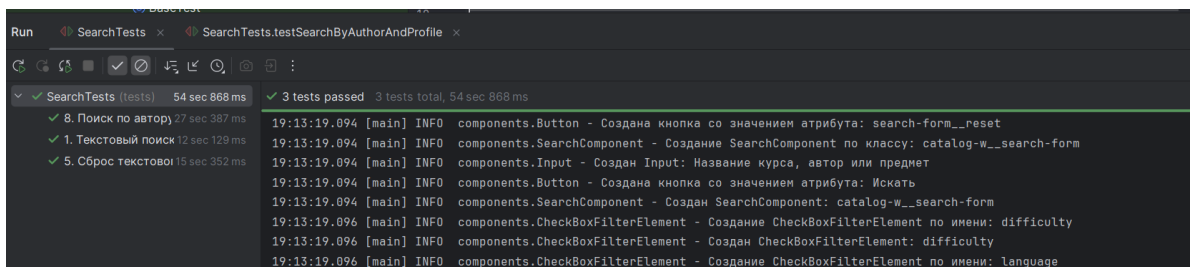


Рисунок 28 – Результаты первого блока

Результаты выполнения второй группы тестов представлены на рис. 29

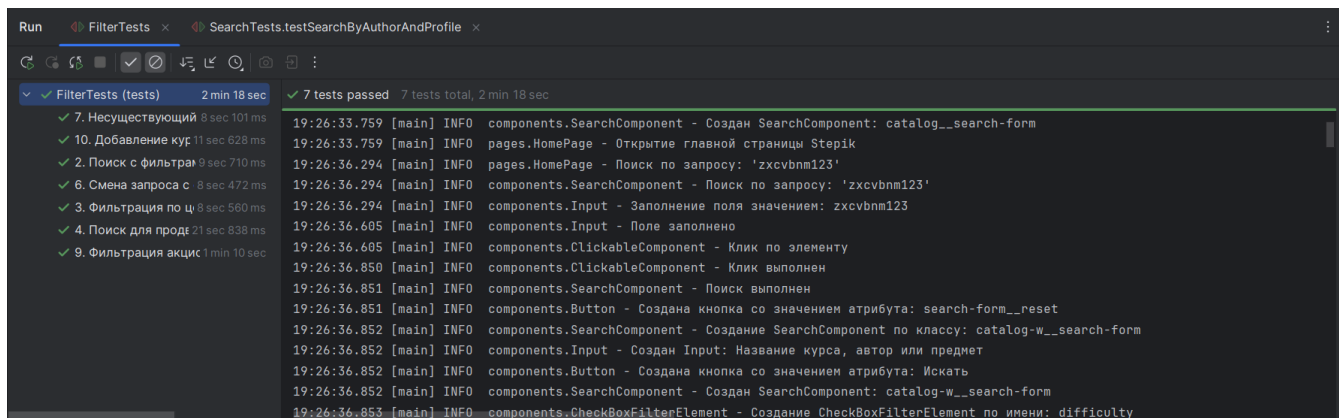


Рисунок 29 – Результаты второго блока

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет components (UI элементы):

1. BaseComponent: Абстрактный базовый класс, представляющий собой обертку над SeleniumElement. Содержит общую логику поиска элементов по XPath [5] и методы доступа к ним.
2. ClickableComponent: Интерфейс, определяющий стандартное поведение для кликабельных элементов с обработкой ожидания видимости и логированием ошибок.
3. Button: Специализированный класс для работы с кнопками. Реализует интерфейс ClickableComponent. Содержит методы создания кнопки по названию (byName) и по CSS-классу (byClass).
4. Input: Специализированный класс для работы с полями ввода. Содержит методы создания поля по placeholder (byPlaceholder), заполнения поля (fill) и получения значения (getValue).
5. CheckBox: Специализированный класс для работы с чекбоксами. Реализует интерфейс ClickableComponent. Содержит метод создания чекбокса по значению data-qa-value (byValue).
6. CheckBoxFilterElement: Класс для работы с группой чекбоксов внутри блока фильтрации. Содержит методы создания фильтра по имени (byName) и получения чекбокса по значению внутри группы (getCheckBoxByValue).

7. SearchComponent: Составной компонент, объединяющий строку поиска и кнопку отправки запроса. Содержит методы создания по CSS-классу (byClass), выполнения поиска (search) и получения текущего значения поля (getCurrentInputValue).
8. FilterComponent: Составной компонент, инкапсулирующий логику работы со всеми фильтрами на странице поиска: фильтрация по цене, сложности, языку и типу курса. Содержит методы применения чекбоксов (filterByCheckBox), ценового фильтра (filterByMinPrice, filterByMaxPrice), тумблеров (filterByToggler) и проверки состояния чекбокса (isCheckBoxSelected).
9. PriceFilterElement: Компонент для работы с ценовым фильтром. Содержит поля ввода минимальной и максимальной цены. Содержит методы получения полей ввода (getminValueInput, getMaxValueInput).
10. ToggleFilterElement: Класс для работы с тумблерами-переключателями. Содержит методы создания по имени (byName), переключения (toggle) и проверки активности (isActive).
11. CourseCardItem: Класс для работы с отдельной карточкой курса в поисковой выдаче. Реализует методы добавления в избранное (addToWishlist), клика по первому автору (clickFirstAuthor), получения заголовка (getTitle), проверки старой цены (hasOldPrice) и проверки наличия в избранном (isFavorite).
12. CourseCardText: Класс для работы с текстовым элементом внутри карточки курса (название или автор). Реализует интерфейс ClickableComponent. Содержит метод получения текста (getText).

Пакет pages (Page Objects):

1. BasePage: Абстрактная основа для всех страниц, содержащая общие параметры ожидания и методы инициализации.
2. HomePage: Описывает главную страницу каталога и иницирует процесс поиска.

3. `SearchResultsPage`: Центральный класс, управляющий результатами поиска, применением фильтров и навигацией по страницам (пагинацией).

4. `CoursePage` и `ProfilePage`: Классы для взаимодействия со страницей конкретного курса и профилем автора соответственно.

Пакет `helpers` и `tests`:

1. `WindowHandler`: Вспомогательный класс для управления окнами и вкладками браузера.

2. `BaseTest`: Класс конфигурации, отвечающий за настройку драйвера перед запуском тестов и закрытие сессии после их завершения.

3. `SearchTests`: Основной тестовый класс, содержащий сценарии текстового поиска, проверки пустых запросов, поиска со спецсимволами и навигации по результатам выдачи.

4. `FilterTests`: Тестовый класс, объединяющий сценарии проверки различных фильтров (цена, язык, сложность, акции). Вынесение этих тестов в отдельный класс позволило логически разделить функционал поиска и фильтрации.

5. `TestConstants`: Класс-хранилище статических констант. В него вынесены все «магические числа» (таймауты, ожидаемые тексты, тестовые наборы данных), что обеспечивает удобство редактирования тестов без изменения программной логики.

2.2. UML-диаграмма классов

На рисунке (см. рис. 30) приведена UML-диаграмма, отображающая структуру классов, их иерархию наследования, реализацию интерфейсов и взаимосвязи между компонентами и страницами.

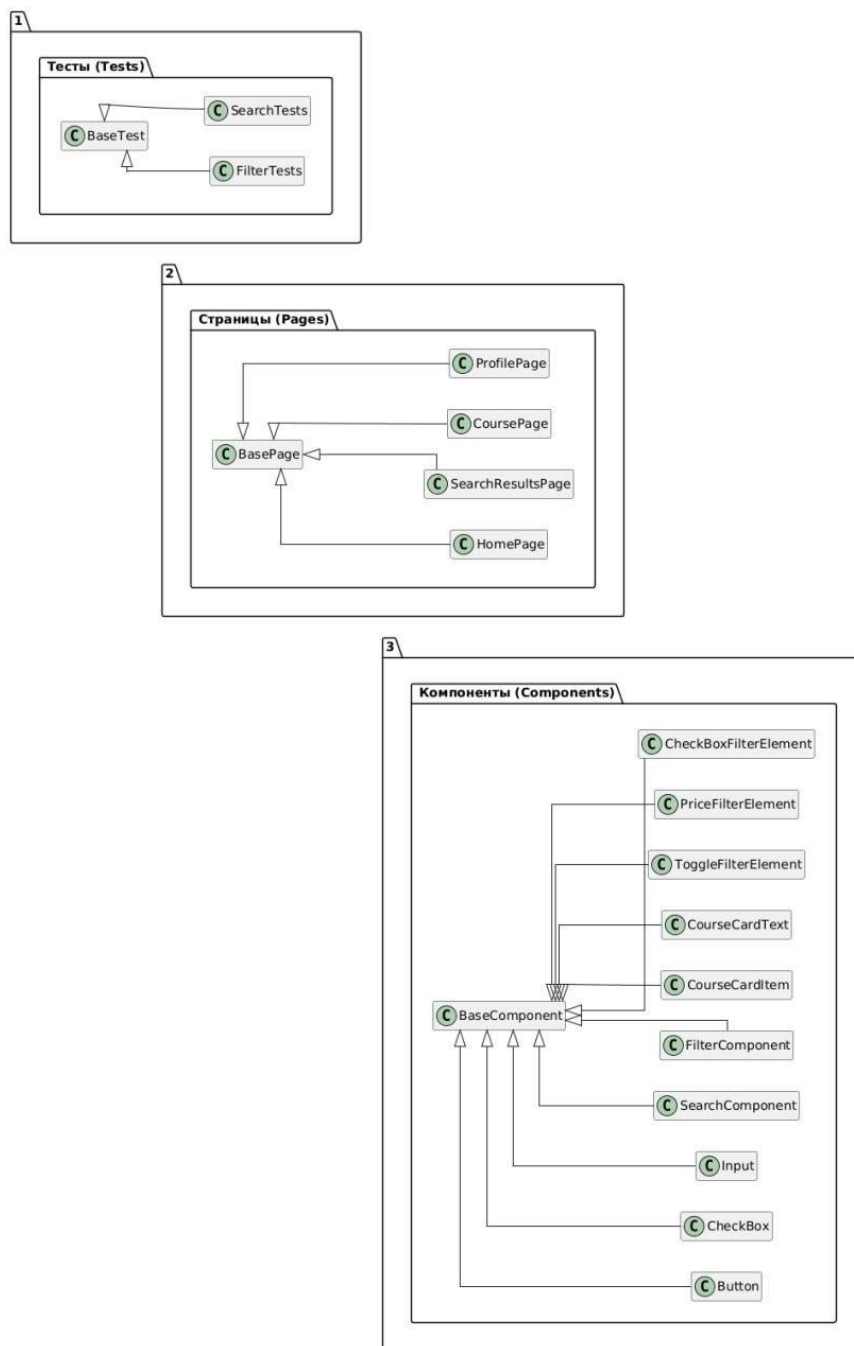


Рисунок 30 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Взаимодействие между классами построено по иерархическому принципу. Тестовые методы обращаются к методам классов страниц (Page Objects). Страницы, в свою очередь, делегируют управление специфическими блоками интерфейса составным компонентам (Components).

Например, при вызове метода `applyMinPriceFilter()` в классе `SearchResultsPage` происходит обращение к вложенному объекту `FilterComponent`, который взаимодействует с объектом `Input`. Такой подход позволяет изменять локаторы элементов в одном месте (в классе компонента) без необходимости править код тестов.

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В рамках работы над проектом автоматизации тестирования платформы Stepik мной был реализован блок тестовых сценариев (№6–10), проведена оптимизация архитектуры фреймворка и внедрена система обработки динамических элементов.

Конкретные задачи, выполненные лично:

1. Реализация сложных тестовых сценариев (№6–10)

Мной были разработаны и внедрены автоматизированные тесты, проверяющие комплексную логику работы фильтров и навигации:

Смена запроса с сохранением фильтра: реализована логика проверки персистентности (сохранения) выбранных фильтров при выполнении повторного поиска с новым текстовым запросом.

Обработка негативных сценариев: реализован тест на ввод несуществующего запроса, проверяющий корректность отображения заглушки «Ничего не найдено» при активных фильтрах.

Поиск по автору и переход в профиль: разработан сценарий, имитирующий переход со страницы результатов поиска в профиль преподавателя с последующей проверкой загрузки блока отзывов.

Фильтрация акционных курсов: реализован сложный алгоритм проверки, который обходит список карточек и подтверждает наличие старой (зачеркнутой) цены у курсов с примененным фильтром «Акция».

Добавление в избранное: автоматизирован процесс взаимодействия с компонентом карточки курса для добавления его в список желаемого и проверки изменения состояния иконки.

2. Создание вспомогательного класса WindowHandler

Для повышения надежности тестов, взаимодействующих с несколькими вкладками браузера, мной был выделен отдельный слой логики:

Создан класс WindowHandler в пакете helpers.

Реализован статический метод `switchToWindow(int index)`, инкапсулирующий стандартные средства Selenide для переключения фокуса драйвера.

3. Рефакторинг и работа с «магическими числами»

В целях повышения читаемости и упрощения поддержки кода мной была проведена работа по вынесению всех жестко заданных значений в именованные константы:

В классах `FilterTests` и `SearchTests` созданы блоки `private static final` переменных (например, `SLEEP_TIMEOUT`, `PROMO_SEARCH_LIMIT`, `TARGET_WINDOW_INDEX`, `FIRST_ITEM`).

Это позволило централизованно управлять задержками и лимитами поиска, исключая ошибки при дублировании значений в коде.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе реализации учебной практики был сформирован технологический стек, обеспечивающий высокую надежность и удобство поддержки разрабатываемого решения. В качестве основного языка программирования был выбран Java, что позволило в полной мере реализовать объектно-ориентированный подход и выстроить четкую архитектуру проекта с разделением ответственности между классами страниц, компонентов и тестов. Управление зависимостями и сборка проекта осуществлялись с помощью инструмента Maven.

Для автоматизации взаимодействия с интерфейсом платформы Stepik применялся фреймворк Selenide. Выбор данного инструмента обусловлен его эффективностью при работе с динамическими элементами, а также наличием встроенных механизмов неявных ожиданий, что существенно повышает стабильность тестов. Структура тестовых сценариев и их запуск реализованы на базе библиотеки JUnit 5, а для мониторинга хода выполнения и оперативного выявления ошибок в проект интегрирована система логирования Log4j2.

Применение выбранных технологий позволило создать масштабируемый тестовый фреймворк и успешно автоматизировать проверку ключевых функций поиска и фильтрации каталога Stepik.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

Отзыв руководителя представлен на рисунке (см. рис. 31).

Отзыв

о прохождении учебной практики

Писаренко Александра Андреевна, студентка группы 4382 второго курса бакалавриата Санкт-Петербургского электротехнического университета им В.И. Ульянова (Ленина) “ЛЭТИ”, проходила учебную практику по теме “UI-тестирование” с 06.07.2025 по 18.07.2025 на кафедре МО ЭВМ.

Во время практики Писаренко Александра Андреевна выполнила все поставленные задачи, продемонстрировала владение языком программирования JAVA, умение планировать разработку и работать в команде.

По результатам учебной практики Писаренко Александра Андреевна заслуживает оценки **отлично**.

Подпись руководителя практики:

Ассистент кафедры
МОЭВМ



Шевелева А.М.

17.07.2026

Рисунок 31 – Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики в период была достигнута поставленная цель – разработка и реализация программного комплекса для автоматизированного UI-тестирования модуля поиска и фильтрации онлайн-платформы Stepik на языке Java с использованием фреймворка Selenide.

Ниже приведены краткие результаты выполнения каждой из задач, обозначенных в разделе «Постановка задачи»:

1. Результат по задаче 1 (Анализ предметной области и проектирование сценариев). Проведён детальный анализ пользовательского интерфейса и бизнес-логики модуля поиска и фильтрации Stepik. Составлен и утверждён чек-лист, включающий 10 функциональных UI-сценариев, охватывающих как базовый текстовый поиск и навигацию, так и сложную комбинированную фильтрацию (по цене, языку, уровню сложности, типу программы и наличию сертификата), а также негативные и интерактивные пользовательские сценарии.

2. Результат по задаче 2 (Проектирование архитектуры тестового фреймворка). Спроектирована и реализована гибкая архитектура автотестов на базе паттерна Page Object Model и компонентно-ориентированного подхода. Выделены атомарные и составные UI-компоненты (Button, Input, CheckBox, SearchComponent, FilterComponent, CourseCardItem). Для обработки специфических UI-действий (переключение между вкладками браузера) разработан вспомогательный класс WindowHandler. Подключена система логирования Log4j2.

3. Результат по задаче 3 (Программная реализация автоматизированных тестов). На языке Java с применением Selenide и JUnit 5 программно реализованы все 10 тестовых сценариев, распределённые по тестовым классам SearchTests и FilterTests. Изоляция веб-элементов внутри классов страниц и компонентов обеспечила читаемость и устойчивость кода к изменениям верстки.

4. Результат по задаче 4 (Проведение тестирования и анализ результатов). Все 10 тестовых сценариев успешно пройдены (статус: PASSED). Подтверждена корректность работы поиска по ключевым словам, правильность применения фильтров, сохранение параметров при повторном поиске, корректность обработки пустых выдач и переходов на сторонние вкладки.

5. Результат по задаче 5 (Оформление технической и отчётной документации). Подготовлен полный комплект документации, включая UML-диаграмму классов фреймворка, файл руководства README, спецификацию тестовых сценариев и итоговый отчёт по учебной практике.

Весь разработанный код приведён к единому стандарту оформления, очищен от неиспользуемых фрагментов и задокументирован. Исходные файлы проекта загружены в репозиторий на GitHub [6], что гарантирует сохранность результатов, контроль версий и возможность совместной работы над материалами тестирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fowler, М. PageObject [Электронный ресурс].
URL: <https://martinfowler.com/bliki/PageObject.html> (дата обращения: 07.07.2026).
2. Selenide: Concise UI Tests in Java [Электронный ресурс].
URL: <https://selenide.org/> (дата обращения: 08.07.2026).
3. JUnit 5 User Guide [Электронный ресурс].
URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. Apache Log4j2 Manual [Электронный ресурс]. URL: <https://logging.apache.org/log4j/2.x/manual/index.html> (дата обращения: 10.07.2024).
5. Язык запросов XPath — руководство MSiter [Электронный ресурс].
URL: <https://msiter.ru/tutorials/xpath> (дата обращения: 07.07.2026).
6. <https://github.com/vyixwq/Stepik-practice>